

PROTON SRE

GENEVA BEGINNER GUIDE

Learn the basic components before learning the product names.
Decode Infrastructure Systems and Application Edge SRE work.
Separate current evidence from skills that still need supervision.

START WITH THE TRUTH

A local lab is not Proton production experience.

A role description is not a qualification certificate.

A six-week plan creates evidence; it does not guarantee a job.

SIMON PARRIS | 19 SEPTEMBER 2026

Learning guide, not an employment or work-authorization claim

0

Scope and validity

Use this book to understand Proton SRE work, not to inflate a CV or assume eligibility.

This guide uses Proton's official job pages checked 19 September 2026. It explains the role's systems and vocabulary in beginner language. It does not claim that the user has Proton experience, production Kubernetes experience, on-call history, a Swiss permit or a job offer.

Fact	Meaning
Official role description	Evidence of what Proton asks for in that vacancy
Public project	Evidence that code and documentation are accessible
Local demonstration	Evidence limited to the tested local environment
Supervised lab proof	A stronger claim only after an approved reviewer observes it
Employment	A real contract, not a guide, project or course

The correct learning order is: definitions -> Linux/networking -> containers -> Kubernetes -> CI/IaC -> observability/security -> incident response. Start with the current local lab; add Proton-like tools only when the environment is approved and the result can be reproduced.

Exercise

Can this PDF honestly say you are a Proton SRE?

Worked answer

No. It can help you understand the work and prepare evidence. A job title requires a real employment relationship.

1

What an SRE does

An SRE keeps a useful service reliable by combining software engineering and operations.

A service can be running yet unusable. SRE work makes useful behaviour measurable, failures visible and recovery repeatable.

Question	SRE answer
What is the user trying to do?	Define the important operation and its success condition
How good is it?	Measure an SLI and agree an SLO
What failed?	Use metrics, logs, traces and layered diagnosis
What should happen now?	Follow a runbook and preserve evidence
How do we improve?	Automate safe repetition and write a postmortem

SRE is not only system administration, monitoring, cloud branding or a Kubernetes certificate. Production SRE also includes change risk, communication, access, escalation, capacity, security and the consequences of being wrong.

Exercise

Why should an SRE care about a failed health check if the process is still running?

Worked answer

Because liveness and useful readiness are different. Keeping a process alive can preserve diagnostics while readiness prevents traffic from being sent to an instance that cannot complete its function.

2

Proton's Geneva production context

Translate the role description into systems you can reason about.

Proton's Infrastructure Systems role is listed for Geneva and Paris. The team provides Kubernetes, VM orchestration and bare-metal provisioning, plus DNS, DHCP, source of truth and monitoring. The description refers to thousands of servers, automation for 99.95%+ uptime, monitoring/alerting, security and on-call troubleshooting.

Proton phrase	What it means for a beginner
Kubernetes platform	A control system that schedules and replaces container workloads
VM orchestration	Automated lifecycle and resource management for virtual machines
Bare metal	Physical servers whose hardware and provisioning are part of the work
DNS/DHCP	Naming and network configuration that many services depend on
Source of truth	A trusted record of intended infrastructure state
99.95%+ uptime	A very high target requiring measurement, redundancy and change discipline
On-call	A staffed response process with escalation, access and runbooks

Proton's Application Edge role is related but different: it focuses on traffic management and load balancing in hybrid on-premises/cloud environments. It names TCP/IP, DNS, HTTP/HTTPS, HAProxy/Envoy/Traefik, Kubernetes, Python/Rust/Go, recovery and security.

These descriptions are production context. They are not a promise that a beginner can perform every responsibility after a course.

3

Core definitions

Memorize the concepts that remain useful even when product names change.

Term	Definition
Availability	Successful intended operations divided by total intended operations in a stated window
Reliability	Ability to perform the intended function for a stated time and conditions
Latency	Time taken for an operation; use a defined percentile or summary
SLI	A measured indicator of a user-relevant outcome
SLO	A target for an SLI over a defined window
SLA	An external contractual promise with consequences
Error budget	Permitted unreliability under an SLO
Observability	Understanding internal state from metrics, logs and traces
Toil	Repetitive manual work with little lasting value
Runbook	Steps for a known operational task or failure
Postmortem	Evidence-based record of impact, timeline, cause and prevention

Example

If 22 of 23 scheduled polls succeed during a drill, the measured availability is $22/23 = 95.652\%$ for that short window. It is not a production SLO, and it must not be presented without the numerator, denominator and window.

Exercise

What is an SLO for a Proton-like telemetry service?

Worked answer

For example: 'At least 99% of scheduled telemetry polls succeed during each 30-minute supervised exercise.'
The target is only meaningful after the operation and window are agreed.

4

The infrastructure path

Follow a request from a name to an application and back to evidence.

```
user -> DNS -> load balancer/proxy -> Kubernetes Service -> Pod
| |
+--> metrics/logs/alerts +--> dependency/data store
```

Layer	Beginner question
DNS	Does the name resolve to the intended destination?
Network	Can packets take the permitted route?
Load balancer	Is traffic distributed and is the backend considered ready?
Kubernetes	Is the desired number of Pods running and healthy?
Application	Can the useful operation succeed?
Dependency	Can the application reach the service or data it needs?
Observability	Can an operator prove what happened?

The lowest failing layer is often the fastest path to diagnosis. Do not restart everything first: preserve evidence, identify impact, then take the smallest safe recovery action.

5

Linux, containers and Kubernetes

Understand the building blocks before attempting a cluster.

Linux provides processes, filesystems, permissions, networking and the kernel. A container is an isolated process sharing the host kernel; it is not a VM. An image is a package of an application and runtime filesystem. Docker builds/runs images and Compose connects several services for a lab.

Kubernetes object	Purpose
Node	Machine or VM that runs workloads
Pod	Smallest deployable unit, usually one main container
Deployment	Declares desired stateless Pods and manages rollout/replacement
Service	Stable network identity for a group of Pods
Readiness probe	Whether an instance should receive traffic
Liveness probe	Whether an instance should keep running
Resource request/limit	Scheduling expectation and maximum resource use
RBAC/NetworkPolicy	API permissions and Pod traffic restrictions

Beginner sequence: deploy one image; add probes; add limits; use a restricted service account; observe the rollout; perform a safe rollback. Only use a disposable or explicitly approved environment.

Exercise

Why can readiness fail while liveness remains healthy?

Worked answer

The process can be running while a dependency or useful operation is unavailable. Readiness protects traffic; liveness protects process recovery.

6

IaC, CI/CD, observability and security

Treat every change as a reviewed, measurable and reversible operation.

Area	Core idea	Example tools in Proton-style roles
IaC	Declare infrastructure and review a plan	Terraform/OpenTofu
Configuration	Keep machines in intended state	Ansible, Puppet
CI/CD	Build, test, scan and release changes	GitLab CI, GitHub Actions, Jenkins
GitOps	Git is desired state; an agent reconciles it	ArgoCD
Metrics	Collect numeric signals	Prometheus
Dashboards	Help people inspect trends and state	Grafana
Logs	Search event records	ELK/Kibana or an approved equivalent
Security	Reduce attack and change risk	IAM/RBAC, scans, hardening, network policy

A sensible security gate checks source/dependencies, builds a minimal non-root image, scans it, creates an SBOM, checks secrets and policy, then deploys only to an approved environment. A missing check is an explicit UNKNOWN or a fail-closed block, not an invented pass.

The current lab already demonstrates GitHub Actions, Docker Compose, Prometheus and Grafana. GitLab, Terraform, Ansible, Kubernetes and production security tooling remain next-stage evidence.

7

Map the current lab to Proton

Use the lab as a learning bridge and label every boundary.

Proton theme	Current lab evidence	Still required
Python automation	FastAPI polling, validation and tests	Reviewed operational script on approved host
Metrics	Prometheus counters and Grafana	Alert owner, threshold, routing and runbook
Linux/networking	TCP protocol and local containers	Approved Linux host, route/link diagnosis
Kubernetes	Compose service graph	Deployment, Service, probes, limits and rollback
Infrastructure automation	Documented setup	Idempotent Terraform/OpenTofu or Ansible plan
Incident response	Controlled fault/recovery	Three witnessed drills and postmortems
Production scale	Not demonstrated	Do not claim thousands of servers or Proton systems

Truthful project statement

Built a local hardware-aware reliability lab with C++, Python/FastAPI, Docker Compose, Prometheus and Grafana; implemented a controlled fault and recovery drill with automated tests and documented operational boundaries.

Do not write 'Site Reliability Engineer at Proton', 'production Kubernetes' or '99.95% uptime' unless those statements are separately true and documented.

8

Six-week practice plan

Create early-career evidence without pretending six weeks creates seniority.

Week	Practice	Definition of done
1	Definitions, current Mac lab, SLI/SLO explanation	Run and explain the fault/recovery drill
2	Approved Linux VM/host, processes, ports, routes, systemd	Agent runs within an approved boundary
3	Docker review and Kubernetes probes/limits	Disposable/approved deployment and rollback
4	CI, dependency/image scan, SBOM and secret checks	A safe introduced issue is caught before deploy
5	SLO, logs/metrics and three incident drills	Timeline, impact, cause, recovery and prevention
6	Restore, handover, skills matrix and fit-gated application	Another person reproduces the runbook

Ask the business centre for one approved host or VM, an isolated lab network, one reviewer for one hour each week and confirmation of available Kubernetes, VMware/Hyper-V, GitLab, logs and security tools. IP4IT's public information establishes programme alignment only; equipment, supervision, funding, placement and authorization remain UNKNOWN until confirmed.

Exercise

What is the first action today?

Worked answer

Run the current lab's `make demo`, explain the user impact, record the result, then read Sections 1-7 aloud. Do not install a production tool or touch a shared network as a substitute for understanding.

9

Glossary exercises and interview answers

Use short, precise answers instead of tool-name lists.

Question	Good beginner answer
What is SRE?	Software engineering applied to operating reliable services.
What is DevSecOps?	Security controls integrated into build, deploy and operate workflows.
What is a readiness probe?	Whether an instance should receive traffic.
What is observability?	Using signals to understand internal state.
What is an error budget?	The permitted unreliability under an SLO.
What is a runbook?	Steps for a known task or failure.
What do you currently prove?	A local C++/Python/Docker/Prometheus/Grafana lab and a controlled recovery drill.
What do you not yet prove?	Proton production systems, fleet scale, independent on-call or production Kubernetes.

Proton's application process asks about location, authorization, salary expectations and management responsibility. Answer using your actual facts. Never use a training plan to fill a work-history field.

10

Sources and recheck gates

Use primary sources and recheck volatile role details before applying.

- Proton SRE Infrastructure Systems: Kubernetes, VM orchestration, bare metal, Linux, IaC, Python, HA, Prometheus/Grafana, security and on-call context.
- Proton SRE Application Edge: traffic management, networking, load balancing, Kubernetes, automation and security.
- Proton careers: Geneva headquarters and current employer context.
- CERN DevOps Engineer for Large Scale Compute: adjacent Geneva early-career stack.
- IP4IT and Geneva Business News IT: programme alignment only.
- [Google SRE book](#): general SRE concepts.

Recheck before any consequential action: the vacancy is still open, the exact requirements and deadline, Proton's application fields, the user's Swiss work authorization, and the centre's actual equipment/supervision. This guide does not submit an application, send a message, register, purchase or begin paid work.

Final validity statement

This PDF is a beginner explanation and an evidence plan. It is sensible if it helps you understand and safely demonstrate the basics. It becomes stronger only when a real reviewer observes reproducible work in an approved environment.